

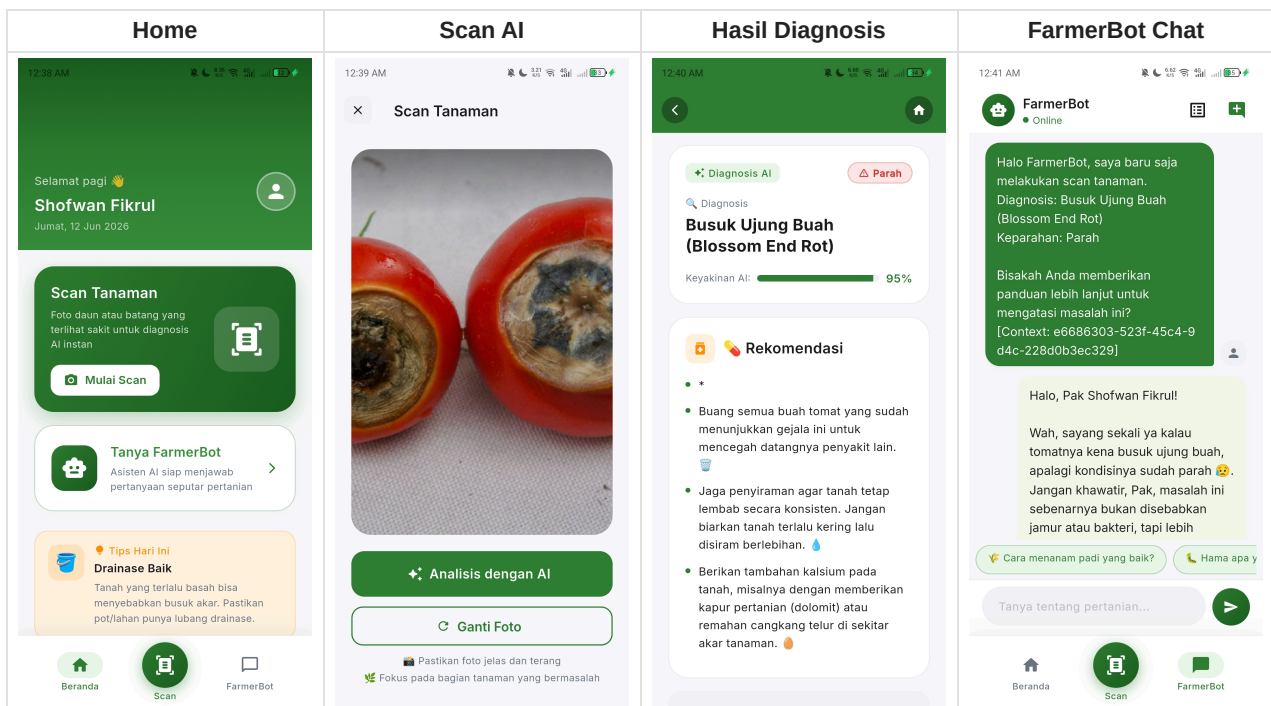
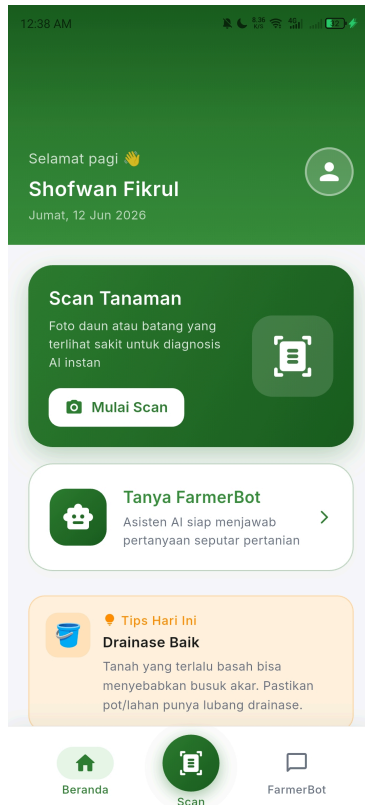


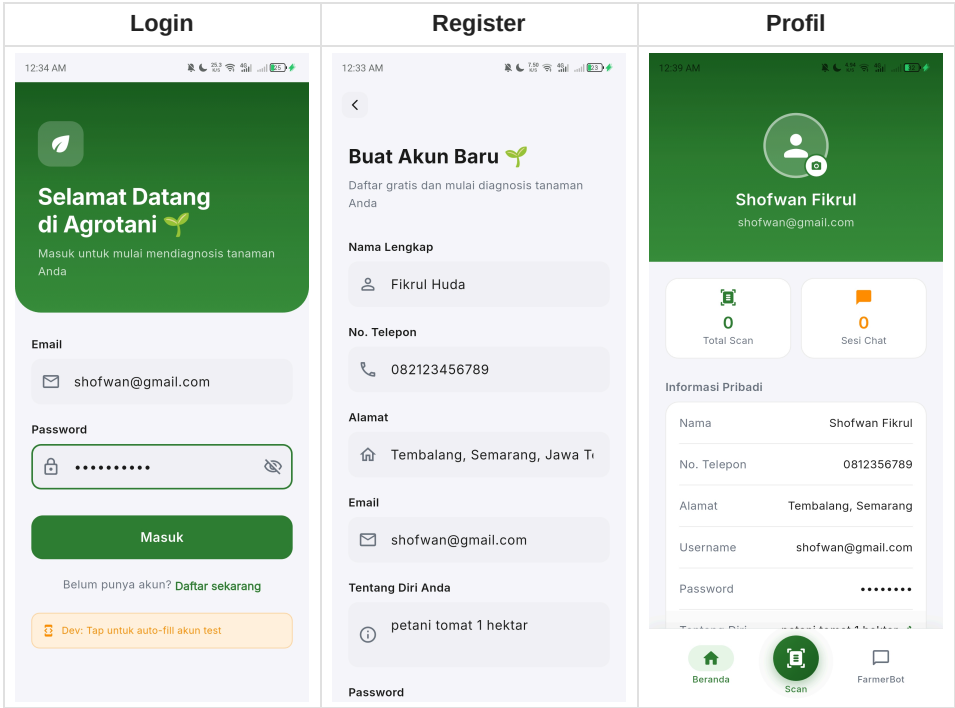
Asisten Cerdas Petani Indonesia — AI-Powered Plant Disease Detection & Advisory

FLUTTER 3.12+ NESTJS 11 G GEMINI AI 2.5 FLASH POSTGRESQL 15 LICENSE UNLICENSED

Agrotani membantu petani Indonesia mendiagnosis penyakit tanaman hanya dengan memfoto daunnya. Didukung oleh **Google Gemini AI**, aplikasi ini memberikan diagnosis, tingkat keparahan, dan rekomendasi penanganan — semuanya dalam Bahasa Indonesia yang mudah dipahami.

Preview Aplikasi





🎯 Apa Itu Agrotani?

Agrotani adalah aplikasi mobile yang dirancang khusus untuk **membantu petani Indonesia** mendeteksi penyakit tanaman secara cepat dan akurat menggunakan kecerdasan buatan (AI).

Masalah yang Diselesaikan

"Tanaman saya sakit, tapi saya tidak tahu penyakitnya apa dan harus ngapain."

Petani sering menghadapi kerugian panen karena **terlambat mendeteksi penyakit** pada tanaman. Akses ke ahli pertanian terbatas, terutama di daerah pedesaan. Agrotani hadir sebagai **solusi digital** yang bisa diakses kapan saja, di mana saja — cukup dengan smartphone.

Cara Kerja

 Foto Tanaman
(3 detik)

→

 AI Analisis
(Gemini AI)

→

 Diagnosis + Rekomendasi
(Bahasa Indonesia)

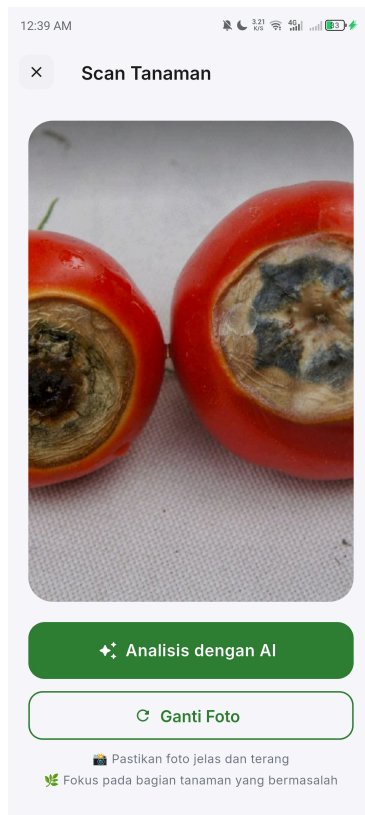
- Foto** — Petani memfoto bagian tanaman yang bermasalah
- Analisis** — Gemini AI menganalisis gambar secara real-time
- Hasil** — Diagnosis penyakit, tingkat keparahan, dan langkah penanganan ditampilkan
- Konsultasi** — Jika masih bingung, petani bisa bertanya lebih lanjut ke **FarmerBot**

🌟 Fitur Utama

📷 Plant Scanner — Deteksi Penyakit via Foto

Ambil foto daun, batang, atau buah tanaman yang terlihat sakit. AI akan menganalisis dan memberikan hasil dalam hitungan detik.

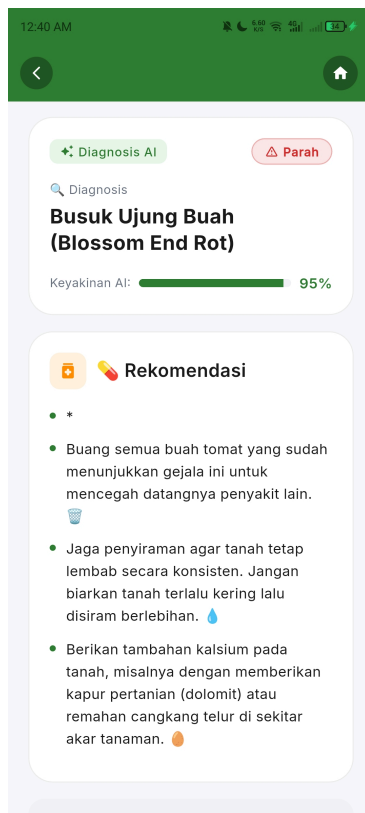
- Mendukung kamera langsung dan galeri foto
- Preview sebelum analisis — pastikan foto jelas
- Kompresi otomatis untuk hemat kuota data
- Riwayat scan tersimpan untuk referensi



AI Diagnosis — Analisis Cerdas

Setiap hasil scan menampilkan:

- **Nama penyakit** yang terdeteksi
- **Tingkat keparahan** (Ringan / Sedang / Parah) dengan visualisasi progress bar berwarna
- **Tingkat keyakinan** AI terhadap diagnosis
- **Penjelasan** dalam bahasa Indonesia sederhana



AI Rekomendasi — Langkah Penanganan

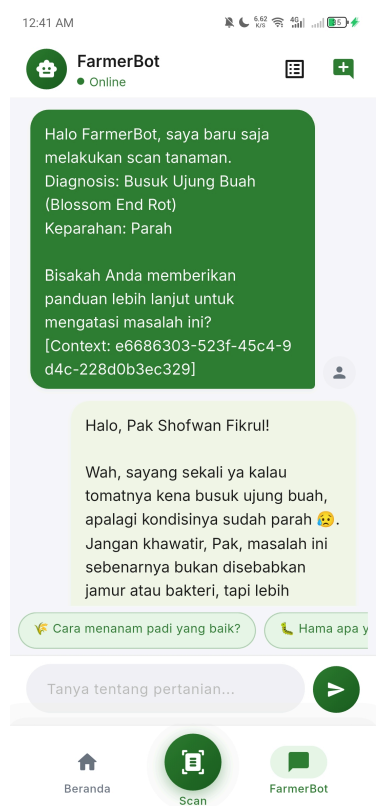
Tidak hanya mendiagnosis — Agrotani juga memberikan **rekomendasi aksi** yang bisa langsung dilakukan petani:

- Langkah penanganan step-by-step
- Jenis obat/pupuk yang sesuai
- Tindakan pencegahan
- Catatan penting

FarmerBot — Chatbot Pertanian

Asisten virtual yang siap menjawab segala pertanyaan seputar pertanian:

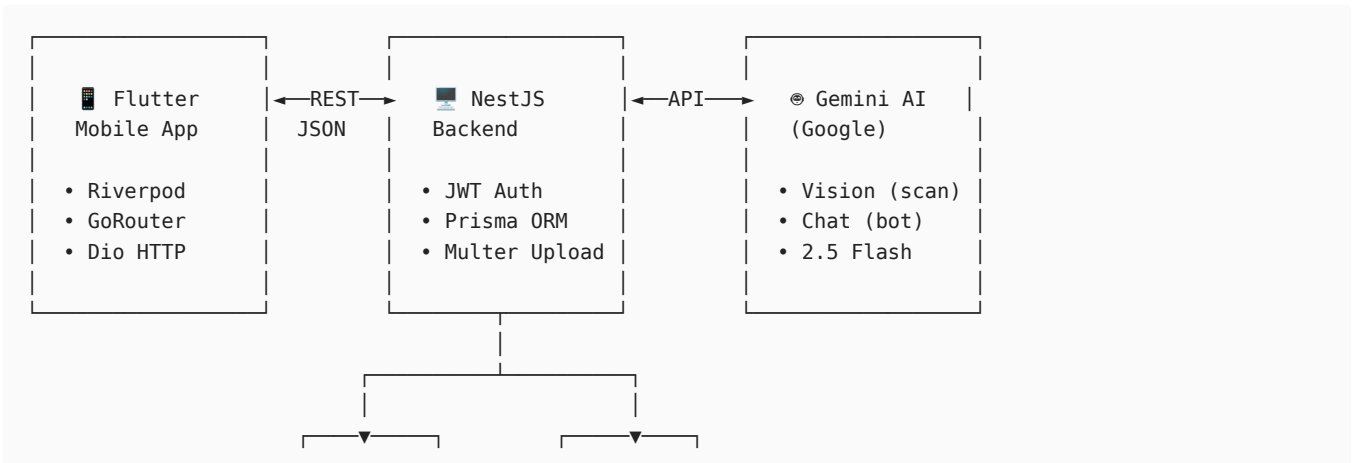
- Tanya jawab bebas dalam Bahasa Indonesia
- Quick reply untuk pertanyaan umum
- Konteks otomatis dari hasil scan — langsung tanya lebih lanjut tentang diagnosis
- Riwayat percakapan tersimpan



Profil & Riwayat

- Riwayat seluruh scan yang pernah dilakukan
- Statistik penggunaan (total scan, sesi chat)
- Sistem feedback untuk membantu AI belajar lebih akurat

Arsitektur Sistem



Mengapa Arsitektur Ini?

Keputusan	Alasan
Monolith NestJS	Sederhana untuk MVP, mudah di-debug, satu deploy
PostgreSQL	Relational, query powerful, gratis, siap scale
Prisma ORM	Type-safe, auto-generate client, migrasi otomatis
JWT Auth	Stateless, standar industri, tidak perlu session server
Gemini 2.5 Flash	Vision-capable, cepat, murah (free tier tersedia)
Flutter + Riverpod	Cross-platform, state management reactive & testable

Tech Stack

Frontend (Mobile App)

Teknologi	Versi	Kegunaan
Flutter	3.12+	Framework UI cross-platform (Android & iOS)
Dart	3.x	Bahasa pemrograman
Riverpod	3.3	State management (reactive, testable)
GoRouter	15.1	Navigasi & routing deklaratif
Dio	5.9	HTTP client dengan interceptor JWT
Flutter Secure Storage	9.2	Penyimpanan token yang terenkripsi
Image Picker	1.1	Akses kamera & galeri

Backend (API Server)

Teknologi	Versi	Kegunaan
NestJS	11.x	Framework backend Node.js (modular, TypeScript)
TypeScript	5.7	Static typing untuk keamanan kode
Prisma	5.22	ORM type-safe untuk PostgreSQL
PostgreSQL	15	Database relasional
Passport + JWT	—	Autentikasi stateless
Multer	2.1	File upload handler
Bcrypt	6.0	Password hashing
Google Generative AI SDK	0.24	Integrasi Gemini API

Infrastructure

Teknologi	Kegunaan
Docker Compose	Menjalankan PostgreSQL secara containerized
Git	Version control

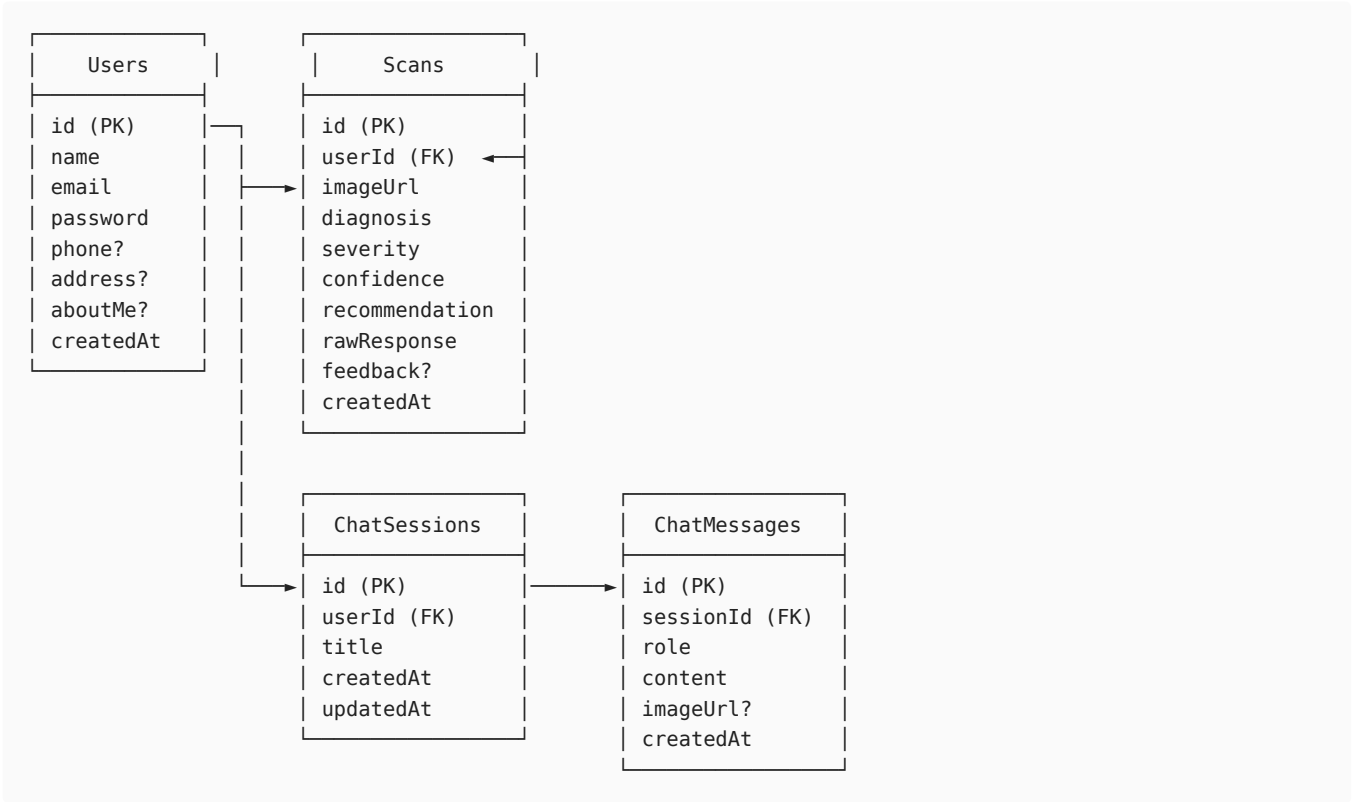
Struktur Proyek

Agrotani/

```
├── app/                                     # Flutter Mobile App
│   ├── lib/
│   │   ├── main.dart                       # Entry point
│   │   ├── app.dart                       # Root widget + MaterialApp
│   │   ├── core/
│   │   │   ├── constants/                 # App-wide constants
│   │   │   ├── network/                  # Dio HTTP client + interceptors
│   │   │   ├── router/                   # GoRouter + auth redirect
│   │   │   ├── services/                 # Token storage service
│   │   │   ├── theme/                    # Colors, typography, theme data
│   │   │   └── widgets/                  # Shared widgets (bottom nav shell)
│   │   └── features/
│   │       ├── auth/                     # Login, register, splash, welcome
│   │       │   ├── data/                 # Models (UserModel), repositories
│   │       │   ├── providers/            # AuthNotifier (Riverpod)
│   │       │   └── screens/              # Login, Register, Splash, Welcome
│   │       ├── chat/                     # FarmerBot chatbot
│   │       │   ├── data/                 # Models, repositories
│   │       │   ├── providers/            # ChatNotifier
│   │       │   └── screens/              # ChatScreen
│   │       ├── home/                     # Dashboard / beranda
│   │       │   └── screens/              # HomeScreen
│   │       ├── profile/                  # Profil pengguna
│   │       │   └── screens/              # ProfileScreen
│   │       ├── scan/                    # Plant scanner
│   │       │   ├── data/                 # ScanResultModel, repository
│   │       │   ├── providers/            # ScanNotifier
│   │       │   └── screens/              # ScanScreen, ScanResultScreen
│   └── api/                              # NestJS Backend
│       ├── src/
│       │   ├── main.ts                   # Bootstrap, CORS, validation pipe
│       │   ├── app.module.ts             # Root module
│       │   ├── auth/                     # Authentication module
│       │   │   ├── auth.controller.ts    # POST /login, /register, GET /profile
│       │   │   ├── auth.service.ts       # Bcrypt, JWT sign/verify
│       │   │   ├── jwt.strategy.ts        # Passport JWT strategy
│       │   │   └── dto/auth.dto.ts        # Login & Register DTOs
│       │   ├── scan/                     # Plant scan module
│       │   │   ├── scan.controller.ts     # POST /analyze, GET /history, /:id
│       │   │   ├── scan.service.ts        # Gemini call, response parsing
│       │   │   └── dto/feedback.dto.ts    # Feedback validation
│       │   ├── chat/                     # Chat module
│       │   │   ├── chat.controller.ts     # POST /send, GET /sessions, /messages
│       │   │   ├── chat.service.ts        # Gemini chat, history management
│       │   │   └── dto/                   # SendMessage DTO
│       │   ├── gemini/                   # Shared AI service
│       │   │   ├── gemini.module.ts       # Gemini Vision + Chat integration
│       │   │   └── gemini.service.ts
│       │   ├── prisma/                   # Database service
│       │   │   ├── prisma.module.ts       # Global module
│       │   │   └── prisma.service.ts      # PrismaClient wrapper
│       │   └── common/
│       │       ├── guards/                # JWT auth guard
│       │       └── decorators/            # @CurrentUser() decorator
│       ├── prisma/
│       │   └── schema.prisma              # Database schema (4 models)
│       ├── uploads/                      # Uploaded plant images
│       ├── docker-compose.yaml            # PostgreSQL container
│       └── .env                           # Environment variables
├── Docs
│   ├── MVP.md                            # MVP specification
│   └── design.md                          # UI/UX design review
```

Database Schema

Agrotani menggunakan 4 tabel utama:



API Documentation

Base URL: `http://localhost:3000/api`

Authentication

Method	Endpoint	Auth	Body	Response
POST	/auth/register	✗	{ name, email, password, phone?, address?, aboutMe? }	{ user }
POST	/auth/login	✗	{ email, password }	{ accessToken }
GET	/auth/profile	✓	—	{ id, name, email, phone, ... }

Plant Scan

Method	Endpoint	Auth	Body	Response
POST	/scan/analyze	✓	multipart/form-data { image }	{ id, diagnosis, severity, confidence, recommendation, ... }
GET	/scan/history? limit=10	✓	—	[{ scan }, ...]
GET	/scan/:id	✓	—	{ scan }
POST	/scan/:id/feedback	✓	{ feedback: "accurate" \ "inaccurate" }	{ scan }

FarmerBot Chat

Method	Endpoint	Auth	Body	Response
POST	/chat/send	✔	{ message, sessionId? }	{ sessionId, reply }
GET	/chat/sessions	✔	—	[{ session }, ...]
GET	/chat/:id/messages	✔	—	[{ message }, ...]

Auth: Semua endpoint bertanda ✔ membutuhkan header Authorization: Bearer <token>

🚀 Cara Menjalankan

Prasyarat

- Node.js v18+ — [Download](#)
- Flutter 3.12+ — [Install](#)
- Docker — [Download](#)
- Gemini API Key — [Dapatkan gratis](#)

1. Clone Repository

```
git clone https://github.com/your-username/Agrotani.git
cd Agrotani
```

2. Setup Backend (API)

```
# Masuk ke folder API
cd api

# Install dependencies
npm install

# Jalankan PostgreSQL via Docker
docker compose up -d

# Konfigurasi environment
cp .env.example .env
# Edit .env → isi GEMINI_API_KEY dan JWT_SECRET
```

Isi file .env :

```
DATABASE_URL="postgresql://root:12345@localhost:5432/agrotani_db?schema=public"
JWT_SECRET="your-random-secret-min-32-chars"
JWT_EXPIRES_IN="7d"
GEMINI_API_KEY="your-gemini-api-key"
PORT=3000
```

```
# Jalankan migrasi database
npx prisma migrate dev --name init

# Jalankan server development
npm run start:dev
```

✔ Backend berjalan di http://localhost:3000

3. Setup Frontend (App)

```
# Kembali ke root, masuk ke folder app
cd ../app
```



```
# Install dependencies
flutter pub get

# Jalankan di emulator atau device
flutter run
```

Catatan: Secara default, app terhubung ke `http://192.168.200.253:3000/api`. Untuk mengubahnya:

```
flutter run --dart-define=API_URL=http://YOUR_IP:3000/api
```

Untuk emulator Android, gunakan `http://10.0.2.2:3000/api`

4. Verifikasi

1. Buka app → tampil **splash screen** → lanjut ke **welcome screen**
2. Daftar akun baru → login
3. Dari home, tap "**Scan Tanaman**" → foto tanaman → lihat diagnosis
4. Tap "**FarmerBot**" → tanya tentang pertanian
5. Cek **profil** → lihat riwayat scan

AI & Prompt Engineering

Agrotani memanfaatkan **Google Gemini 2.5 Flash** dengan prompt engineering yang telah dioptimasi khusus untuk konteks pertanian Indonesia.

Diagnosis Prompt

AI diminta memberikan output dalam format terstruktur:

	Diagnosis	→ Nama penyakit yang terdeteksi
	Keparahan	→ Ringan / Sedang / Parah
	Rekomendasi	→ Langkah penanganan step-by-step
	Catatan	→ Hal penting yang perlu diperhatikan

FarmerBot Persona

Kamu adalah FarmerBot, teman ngobrol petani yang ramah.
Jawab pertanyaan tentang pertanian dengan bahasa sederhana.
Berikan jawaban yang praktis dan bisa langsung dilakukan.
Akhiri jawaban dengan 1-2 saran pertanyaan lanjutan.

Prinsip Keamanan AI

-  Jawab hanya dalam Bahasa Indonesia
-  Gunakan bahasa sederhana yang dipahami petani
-  Sertakan disclaimer untuk kasus yang perlu konsultasi ahli
-  Tidak mengarang nama penyakit atau dosis obat
-  Tidak menjawab pertanyaan di luar topik pertanian

Design Philosophy

Desain Agrotani dikembangkan berdasarkan [design review](#) komprehensif dengan prinsip:

Dibuat untuk Petani

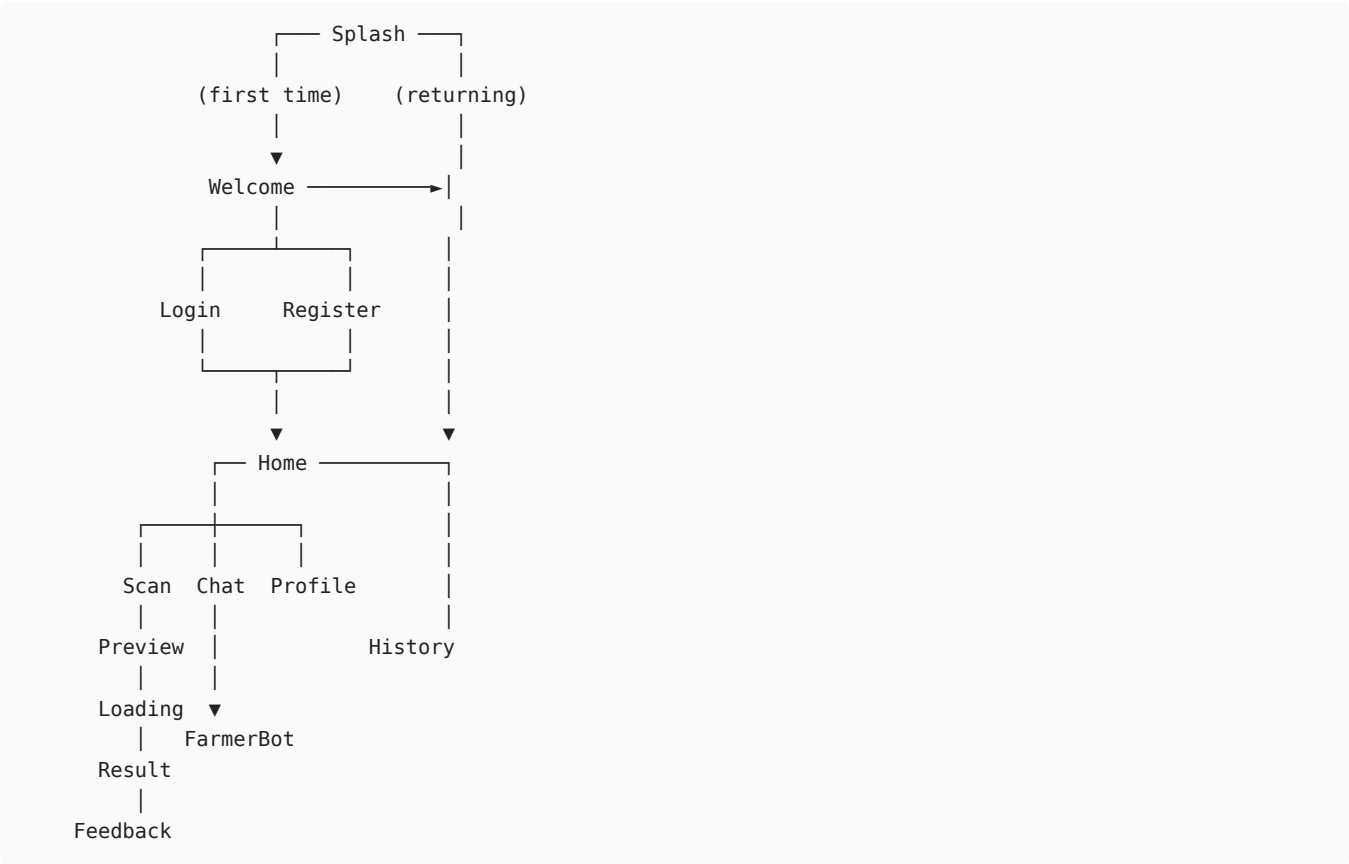
- **Font besar** (minimum 18sp) — mudah dibaca di luar ruangan
- **Tombol besar** (minimum 56dp) — ramah untuk jari kasar

- **Bahasa Indonesia sederhana** — tanpa jargon teknis
- **Ikun > teks** — visual lebih cepat dipahami

Color System

Token	Hex	Kegunaan
● Primary	#2E7D32	Tombol utama, CTA, brand
● Primary Dark	#1B5E20	Header, app bar
● Primary Light	#E8F5E9	Background card
● Accent	#FF8F00	Highlight, badge
● Severity Low	#4CAF50	Ringan (sehat)
● Severity Medium	#FF9800	Sedang (waspada)
● Severity High	#F44336	Parah (bahaya)

User Flow



Ringkasan untuk Stakeholder

Value Proposition

Untuk	Manfaat
Petani	Deteksi penyakit tanaman instan, gratis, tanpa perlu ahli
Penyuluh Pertanian	Tools bantu diagnosis di lapangan, data digital
Dinas Pertanian	Data penyakit tanaman real-time per wilayah (potensi)
Agritech Startup	Platform yang siap dikembangkan ke marketplace & IoT

Potensi Pengembangan

Fase	Fitur	Dampak
MVP 	Scan, Diagnosis, Chat	Validasi konsep
v1.1	Weather integration, Smart Urgency	Peringatan dini
v1.2	WhatsApp Bot, Offline mode	Jangkauan lebih luas
v2.0	Marketplace, Kelompok Tani, IoT	Ekosistem digital pertanian

Key Metrics (Target)

Metrik	Target MVP
Waktu dari foto → diagnosis	< 10 detik
Akurasi diagnosis (feedback user)	> 70%
Jumlah scan per user per minggu	≥ 2
Retention rate (7 hari)	> 40%

Roadmap

Fase 1 — MVP (Sekarang)

-  Plant scanner dengan kamera/galeri
-  AI diagnosis powered by Gemini
-  Rekomendasi penanganan
-  FarmerBot chatbot
-  User authentication (email + password)
-  Riwayat scan
-  Feedback system

Fase 2 — Enhancement

- ☐ Integrasi cuaca (BMKG API) untuk konteks diagnosis
- ☐ Smart Urgency System — peringatan jika penyakit berisiko menyebar
- ☐ Offline scan dengan model TFLite lokal
- ☐ Push notification untuk reminder tindakan
- ☐ Multibahasa (Jawa, Sunda)

Fase 3 — Scale

- ☐ WhatsApp Bot — akses tanpa install app
- ☐ Kelompok Tani — fitur komunitas per daerah
- ☐ Marketplace pupuk & pestisida
- ☐ Dashboard analytics untuk penyuluh
- ☐ IoT sensor integration
- ☐ RAG knowledge base penyakit lokal

Testing

Backend

```
cd api

# Unit tests
npm run test
```

```
# E2E tests
npm run test:e2e

# Test coverage
npm run test:cov
```

Frontend

```
cd app

# Flutter tests
flutter test
```

Manual Testing Checklist

Auth Flow:

- ✓ Register → berhasil buat akun
- ✓ Login → dapat token, masuk ke home
- ✓ Profile → data tampil sesuai
- ✓ Logout → kembali ke welcome

Scan Flow:

- ✓ Foto dari kamera → preview → analisis → hasil
- ✓ Foto dari galeri → preview → analisis → hasil
- ✓ Hasil diagnosis tampil: penyakit, keparahan, rekomendasi
- ✓ Feedback thumbs up/down → tersimpan
- ✓ Riwayat scan muncul di home dan profil

Chat Flow:

- ✓ Kirim pesan → dapat response dari FarmerBot
- ✓ Quick reply buttons → berfungsi
- ✓ Dari hasil scan → "Tanya FarmerBot" → konteks terbawa
- ✓ History chat tersimpan

Dokumentasi Pendukung

Dokumen	Deskripsi
MVP.md	Spesifikasi MVP — scope, timeline, tech stack
design.md	UI/UX review — wireframe, design system, copywriting
implementation.md	Panduan implementasi backend step-by-step
task_list.md	Daftar issue sinkronisasi FE ↔ BE

Cara Berkontribusi

1. **Fork** repository ini
2. Buat **branch** baru: `git checkout -b feature/nama-fitur`
3. **Commit** perubahan: `git commit -m "feat: tambah fitur X"`
4. **Push** ke branch: `git push origin feature/nama-fitur`
5. Buat **Pull Request**

Konvensi Commit

```
feat:    Fitur baru
fix:     Perbaiki bug
docs:    Perubahan dokumentasi
style:   Formatting (tanpa perubahan logika)
```

```
refactor: Refactoring kode
test:      Menambah/memperbaiki test
chore:     Maintenance (dependencies, config)
```

Lisensi

Proyek ini dibuat sebagai bagian dari **tugas akademik / portofolio**. Seluruh kode ditulis secara original.

Developer

Agrotani dikembangkan oleh mahasiswa dengan semangat membantu petani Indonesia melalui teknologi.

 **Agrotani — Karena setiap tanaman berhak mendapat perawatan yang tepat.**

Dibangun dengan  untuk Petani Indonesia